

Sistemas Operativos



Introducción

Qué es un Sistema Operativo

¿Qué es un computador?



- Es un sistema digital compuesto de los siguientes módulos:
 - CPU (procesador)
 - Memoria
 - Interconexión (bus/red)
 - Dispositivos de E/S
- La CPU *manda* y repite constantemente la siguiente secuencia:
 - 1 Traer la **siguiente instrucción máquina** de la Memoria
 - 2 Decodificar la instrucción
 - 3 Obtener los operandos
 - 4 Realizar la operación indicada
 - 5 Guardar el resultado
 - 6 Volver al punto 1
- Observaciones:
 - Las instrucciones deben estar en memoria
 - Los datos de entrada deben estar en memoria
 - Los datos de salida se generan en memoria

¿Qué es un Sistema Operativo?



Es un software cuya misión es facilitar la ejecución de programas de forma eficiente, permitiendo:

- la ejecución simultánea de varios programas
- compartir la memoria entre ellos de forma segura
- sincronizar programas entre sí
- la interacción de los programas con los dispositivos

Para ello el SO:

- Transforma cada recurso hardware (cpu, memoria, disco, etc.) en una versión **virtual** más general, potente y fácil de usar de sí misma.
- Ofrece interfaces (APIs) para usar estos recursos virtualizados, conocidas como *llamadas al sistema*, ofreciendo así distintos **servicios** al programador.

El sistema es considerado un **gestor** de estos recursos virtualizados y un **proveedor de servicios** al programador.

Agenda



1 Servicios del S.O.

2 Cómo actúa el SO

3 POSIX

Virtualización de la CPU



```
#include "common.h"
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "usage: cpu <string>\n");
        exit(1);
    }
    char *str = argv[1];

    while (1) {
        printf("%s\n", str);
        Spin(1);
    }
    return 0;
}
```

Spin: espera activa hasta que pase el número de segundos indicado.

Virtualización de la CPU



Podemos ejecutar el programa en un sistema con una única CPU

```
$ ./cpu A
A
A
A
A
^C
$
```

Virtualización de la CPU



Podemos ejecutar el programa en un sistema con una única CPU

```
$ ./cpu A
A
A
A
A
^C
$
```

También podemos ejecutar varias instancias del programa ...

```
$ ./cpu A & ./cpu B & ./cpu C & ./cpu D &
[1] 2517541
[2] 2517542
[3] 2517543
[4] 2517544
B
A
C
D
C
...
```

Gestión de Procesos



El SO, con ayuda del HW, transforma una CPU en un número aparentemente infinito de CPUs virtuales, generando la ilusión de poder ejecutar simultáneamente varios procesos.

Abstracción del SO: Proceso

Instancia de ejecución de un programa, con recursos asignados (memoria, ficheros, semáforos, ...)

El S.O. ofrece servicios de gestión de procesos, que incluyen:

- Creación de procesos y ejecución de programas
- Planificación de tareas, decidiendo cuándo debe ejecutar cada una y por cuánto tiempo

Virtualización de la memoria



```
#include "common.h"
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    int *p, pid;
    p = malloc(sizeof(int));
    assert(p != NULL);
    pid = (int)getpid();
    printf("(%)d  addr pointed to by p: %p\n", pid, p);
    *p = 0;
    while (1) {
        Spin(1);
        *p = *p + 1;
        printf("(%)d  value of p: %d\n", getpid(), *p);
    }
    return 0;
}
```

Virtualización de la memoria



Podemos ejecutar el programa, no muy interesante

```
$ ./mem
(2592169) addr pointed to by p: 0x5555555592a0
(2592169) value of p: 1
(2592169) value of p: 2
(2592169) value of p: 3
(2592169) value of p: 4
^C
```



Virtualización de la memoria

Podemos ejecutar el programa, no muy interesante

```
$ ./mem
(2592169) addr pointed to by p: 0x5555555592a0
(2592169) value of p: 1
(2592169) value of p: 2
(2592169) value of p: 3
(2592169) value of p: 4
^C
```

Pero si lanzamos dos instancias ...

```
$ ./mem & ./mem &
[1] 2594876
[2] 2594877
(2594876) addr pointed to by p: 0x5555555592a0
(2594877) addr pointed to by p: 0x5555555592a0
(2594876) value of p: 1
(2594877) value of p: 1
(2594876) value of p: 2
(2594877) value of p: 2
(2594876) value of p: 3
(2594877) value of p: 3
...
```

Gestión de Memoria



El SO, con ayuda del HW, virtualiza la memoria física:

- Cada proceso accede a su propio espacio de memoria virtual, mapeando de alguna forma estos espacios virtuales sobre la memoria física del computador.
- Las referencias a memoria de un programa en ejecución no afectan a los espacios de direcciones de los otros procesos del sistema ni al del propio SO.

El S.O. ofrece los siguientes servicios:

- Gestión del espacio de direcciones del proceso: añadir y eliminar regiones y seleccionar su contenido.
- **Reparto de memoria física y protección**
 - El SO mapea los distintos espacios virtuales de direcciones sobre rangos disjuntos de direcciones físicas, usando la memoria física como una caché de los espacios virtuales de los procesos.
 - Otorga permisos de acceso distintos a las regiones virtuales.
- Gestión de la memoria libre, no asignada aún a ningún proceso.



Concurrencia

```
#include "common.h"
#include "common_threads.h"
#include <stdio.h>
#include <stdlib.h>
volatile int counter = 0;
int loops;

void *worker(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        counter++;
    }
    return NULL;
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "usage: threads <loops>\n");
        exit(1);
    }
    loops = atoi(argv[1]);
    pthread_t p1, p2;
    printf("Initial value : %d\n", counter);
    Pthread_create(&p1, NULL, worker, NULL);
    Pthread_create(&p2, NULL, worker, NULL);
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("Final value   : %d\n", counter);
    return 0;
}
```

Concurrencia



Podemos ejecutar el programa para 1.000 iteraciones

```
$ ./threads 1000  
Initial value : 0  
Final value   : 2000
```

Concurrencia



Podemos ejecutar el programa para 1.000 iteraciones

```
$ ./threads 1000  
Initial value : 0  
Final value   : 2000
```

¿Y para 100.000 iteraciones?

```
$ ./threads 100000  
Initial value : 0  
Final value   : 135117
```

Concurrencia



Podemos ejecutar el programa para 1.000 iteraciones

```
$ ./threads 1000  
Initial value : 0  
Final value   : 2000
```

¿Y para 100.000 iteraciones?

```
$ ./threads 100000  
Initial value : 0  
Final value   : 135117
```

¿Y si repetimos? ... ¡¿Qué está pasando?!

```
$ ./threads 100000  
Initial value : 0  
Final value   : 156096
```

Comunicación y sincronización



El SO, con ayuda del HW, ofrece mecanismos de sincronización de tareas concurrentes:

- Creación de secciones críticas
- Mecanismos de bloqueo condicional para implementar esperas pasivas
- Mecanismos para desbloquear a tareas bloqueadas en uno de estos mecanismos

Asimismo, el SO ofrece mecanismos para comunicar procesos

- Por memoria, haciendo una región accesible en ambos procesos
- Sockets
- Paso de mensajes
- Pipes
- Sockets

Persistencia



```
#include <stdio.h>
#include <unistd.h>
#include <assert.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <string.h>

int main(int argc, char *argv[]) {
    int fd = open("/tmp/file", O_WRONLY|O_CREAT|O_TRUNC,
        S_IRUSR|S_IWUSR);
    assert(fd >= 0);
    char buffer[20];
    sprintf(buffer, "hello world\n");
    int rc = write(fd, buffer, strlen(buffer));
    assert(rc == (strlen(buffer)));
    fsync(fd);
    close(fd);
    return 0;
}
```

Persistencia



Podemos ejecutar el programa

```
$ cat /tmp/file
cat: /tmp/file: No such file or directory
$ ./io
$ cat /tmp/file
hello world
```

El SO permite almacenar información en disco de manera persistente.

Gestión del sistema de ficheros



El S.O. ofrece la abstracción de un sistema de ficheros:

- Un modelo simple de fichero al programador y un API sencillo para usarlos
- Ficheros especiales (Directorios) que permiten agrupar ficheros, formando un árbol de directorios.
 - API específico para directorios

y se encarga de forma transparente de:

- Asignar espacio de disco a los ficheros
- Localizar los ficheros en disco a partir de su nombre/ruta
- Gestionar espacio libre en disco
- Identificar los ficheros y sus permisos/propietario
- Permitir accesos coordinados desde varios procesos
- Recuperar información ante problemas inesperados (cortes de luz, etc)

Gestión de la E/S



El S.O. ofrece los siguiente servicios:

- Drivers que se encargan de la gestión de bajo nivel de los dispositivos
- Un interfaz sencillo para el uso de dispositivos
- Un mecanismo de carga de drivers
- Facilidades para la implementación de drivers
- Un esquema jerárquico para el diseño
- Un sistema de cache asociado a los dispositivos para mejorar el rendimiento

Objetivos de la asignatura de SO



- Conocer los servicios que el SO ofrece al programador y aprender a utilizarlos correctamente
- Familiarizarse con el API de un sistema POSIX para usar estos servicios
- Comprender los mecanismos utilizados por el SO para ofrecer estos servicios
 - Está claro por qué ofrece el sistema estos servicios: para facilitar el manejo del computador
 - Nos interesa comprender **cómo** los implementa el SO

Objetivos de la asignatura de SO



- Conocer los servicios que el SO ofrece al programador y aprender a utilizarlos correctamente
- Familiarizarse con el API de un sistema POSIX para usar estos servicios
- Comprender los mecanismos utilizados por el SO para ofrecer estos servicios
 - Está claro por qué ofrece el sistema estos servicios: para facilitar el manejo del computador
 - Nos interesa comprender **cómo** los implementa el SO

Es fundamental **trabajar las prácticas**
para alcanzar estos objetivos

Agenda

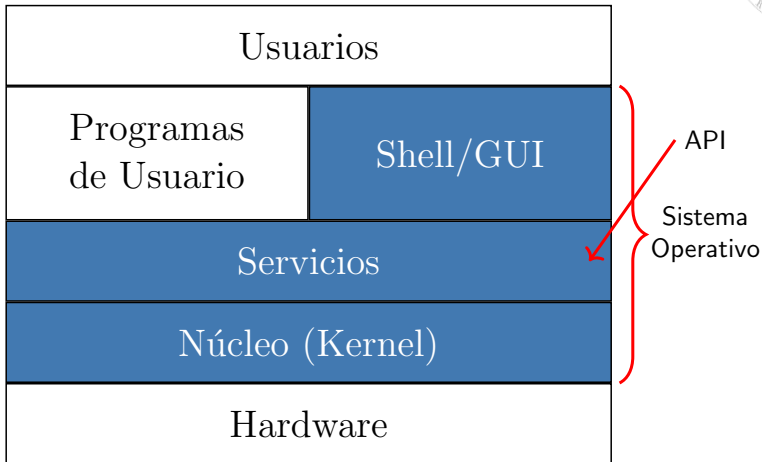


1 Servicios del S.O.

2 Cómo actúa el SO

3 POSIX

El SO es software



El SO es software



Un programa más al fin y al cabo:

- ¿Cómo puede controlar y restringir el acceso a la memoria y los dispositivos de los demás programas?
- ¿Cómo comienza a ejecutarse ese programa sin los servicios del SO?
- ¿Cómo puede un programa en ejecución obtener los servicios del SO, que en ese momento no se está ejecutando?
- ¿No es demasiada sobrecarga controlar cada acceso a memoria?

Soporte HW necesario



- Distintos **modos de ejecución** del procesador
 - Modo usuario (ring level 3 en x86) y Modo kernel (ring level 0 en x86)
- **Interrupciones y excepciones**: permiten ejecutar una rutina registrada cambiando a modo kernel simultaneamente.
 - Excepción de Reset para el arranque del sistema
 - Interrupciones software, para la implementación de las **llamadas al sistema** que ofrecen los servicios al programador
 - Timer HW para generar **interrupciones periódicas** que permitan al SO intervenir periódicamente

El retorno de interrupción reanuda la ejecución en el punto interrumpido restaurando el modo de ejecución.

- Soporte HW para la traducción de direcciones y el control de acceso a memoria (*Memory Management Unit*, **MMU**)

Arranque del SO



- El HW se inicia con una excepción especial: *Reset*
- El vector de reset y la RTI correspondiente están en memoria no volátil (ROM, Flash)
 - El código de la ROM/Flash (BIOS en PC) se encarga de buscar un SO (o gestor de arranque) en los posibles dispositivos de arranque
 - Si lo encuentra copia el primer sector en memoria
 - Salta a la zona de memoria donde lo ha copiado
 - El código copiado se encarga de cargar todo el S.O. en memoria
 - Finalmente comienza la ejecución del S.O.

Agenda



1 Servicios del S.O.

2 Cómo actúa el SO

3 POSIX

Estándar POSIX



- POSIX: Portable Operating System Interface for uniX
 - Estándar de sistemas operativos UNIX de IEEE.
 - NO es una implementación. Sólo define una interfaz.
- Objetivo: portabilidad de las aplicaciones entre diferentes plataformas y sistemas UNIX
- Compuesto o dividido en varios estándares
 - 1003.1 Servicios básicos del SO
 - 1003.1a Extensiones a los servicios básicos
 - 1003.1b Extensiones de tiempo real
 - 1003.1c Extensiones de procesos ligeros
 - 1003.2 Shell y utilidades
 - 1003.2b Utilidades adicionales

Características de POSIX



- Nombres de funciones cortos y en letras minúsculas
 - fork
 - read
 - close
 - ...
- Las funciones normalmente devuelven 0 en caso de éxito o -1 en caso de error
 - Variable **errno**
- Recursos gestionados por el sistema operativo se referencian mediante descriptores